

# Kids Typing Game

## Simple Development Plan and Coding Agent Context

A small browser game where a child presses keyboard keys, sees colorful reactions, and hears the key spoken aloud.

|              |                                                               |
|--------------|---------------------------------------------------------------|
| Project Type | Private/local browser app for one child                       |
| Technology   | Next.js + React + TypeScript + Tailwind CSS                   |
| Backend      | None. Browser-only for version 1                              |
| Main Goal    | Press any key, show it, speak it, and make the screen playful |

## 1. What I Want To Build

Build a very simple typing game for my kid. The app should run in the browser. When she presses any key on the keyboard, the app should show that key clearly and speak the key aloud. The app should also use bright colors, emojis, and small animations so the experience feels like entertainment, not study.

- The app is only for family/private use for now.
- No login, no database, no payment, no analytics, no complex admin panel.
- The first version should be small, clean, and easy to improve later.
- The coding agent should build the same structure in a maintainable way, not a messy one-file experiment.

## 2. Recommended Technology

- **Framework:** Next.js with App Router
- **Language:** TypeScript
- **UI:** React components with Tailwind CSS
- **Voice:** Browser SpeechSynthesis API
- **Storage:** No storage needed for version 1
- **Deployment:** Local browser first; optionally Vercel later

Reason: this keeps the project simple, browser-based, easy to run, and familiar for a React/Next.js development workflow.

## 3. Version 1 Scope

- One main screen only.
- The child presses any keyboard key.
- The app displays the key in very large text.
- The browser says the key aloud.
- The screen changes random bright/pastel colors.
- The app shows a random emoji and friendly message.
- The app keeps a recent history of the last 10 keys.
- Add simple buttons: Mute/Unmute, Clear, and Fullscreen.

## 4. Core User Flow

1. Open the app in the browser.
2. The screen says: Press any key!
3. Child presses A.
4. App displays A in the center.
5. Browser says: A.
6. Background color changes.
7. Emoji and message appear.
8. Recent history updates.
9. Child continues pressing more keys.

## 5. Key Behavior Rules

- The app should say the actual key pressed.
- If she presses A, say A.
- If she presses D, say D.
- If she presses 1, say one.
- If she presses dot, say dot.
- If she presses slash, say slash.
- Do not say random different letters.
- Special keys should have friendly names, such as Space, Enter, Backspace, Shift, and Escape.

## 6. Example Key Mapping

| Pressed Key | Displayed Text | Spoken Text |
|-------------|----------------|-------------|
| A           | A              | A           |
| D           | D              | D           |
| 1           | 1              | one         |
| 0           | 0              | zero        |
| .           | .              | dot         |
| ,           | ,              | comma       |
| ;           | ;              | semicolon   |
| /           | /              | slash       |
| Space       | Space          | space       |
| Enter       | Enter          | enter       |
| Backspace   | Backspace      | backspace   |

## 7. Suggested Folder Structure

```
kids-typing-game/  
  src/  
    app/  
      page.tsx  
      layout.tsx  
      globals.css  
    components/  
      TypingGame.tsx  
      BigKeyDisplay.tsx  
      RecentKeys.tsx  
      ControlButtons.tsx  
    lib/  
      getSpeechText.ts
```

```
getDisplayText.ts  
random.ts  
constants.ts
```

## 8. Component Responsibilities

### TypingGame.tsx

- Main game component.
- Listens to keyboard events.
- Stores current key, spoken text, colors, emoji, message, mute state, and recent key history.
- Calls speech function when a key is pressed.

### BigKeyDisplay.tsx

- Displays the big key in the center.
- Displays emoji, message, and spoken text.
- Adds pop/bounce animation for every key press.

### RecentKeys.tsx

- Shows the last 10 keys pressed.
- Should be visually simple and readable.

### ControlButtons.tsx

- Mute/Unmute button.
- Clear button.
- Fullscreen button.

## 9. Required Utility Files

### getSpeechText.ts

Converts a keyboard event key into child-friendly spoken text. Letters should become uppercase letter names. Numbers should become words. Punctuation should become readable words.

### getDisplayText.ts

Converts the pressed key into clean display text. Example: space becomes Space, Escape becomes Esc, Control becomes Ctrl.

### random.ts

Contains a reusable getRandomItem helper for colors, emojis, and messages.

### constants.ts

Contains arrays for background colors, letter colors, emojis, and encouragement messages.

## 10. UI Direction

- Full-screen playful layout.
- Large central card with rounded corners.

- Very large key text.
- Bright pastel background colors.
- Big emoji reaction.
- Simple encouragement messages such as Great job, Wow, Nice, Super, Keep going.
- No complicated navigation.
- Desktop keyboard-first experience.
- Should also look acceptable on tablets/laptops.

## 11. Development Phases

### Phase 1 - Basic Working Game

- Create Next.js app.
- Create main TypingGame component.
- Detect keydown events.
- Show the pressed key.
- Speak the key using SpeechSynthesis API.

### Phase 2 - Kid-Friendly Visuals

- Add random background colors.
- Add random letter colors.
- Add emoji reaction.
- Add random encouraging message.
- Add pop/bounce animation.

### Phase 3 - Controls and Polish

- Add Mute/Unmute.
- Add Clear.
- Add Fullscreen.
- Keep recent key history.
- Improve special key handling.

## 12. Acceptance Criteria

1. npm run dev starts the app successfully.
2. Opening the browser shows Press any key.
3. Pressing A displays A and says A.
4. Pressing D displays D and says D.
5. Pressing 1 displays 1 and says one.
6. Pressing . displays . and says dot.
7. Pressing Space displays Space and says space.
8. Each key press changes colors.
9. Each key press shows a random emoji/message.
10. Last 10 keys are visible.
11. Mute button stops voice.
12. Clear button resets the screen/history.
13. Fullscreen button enters fullscreen where supported.
14. No backend, database, or authentication is added.

## 13. What Not To Build Now

- Login or user accounts.
- Database.
- Admin panel.
- Analytics.
- Payment or subscription.
- Complex game levels.
- AI features.
- Leaderboard.
- Backend API.

## 14. Future Ideas After Version 1

- Alphabet challenge mode: show a target letter and ask the child to press it.
- A for Apple mode: say A for Apple, B for Ball, C for Cat.
- Animal theme: A for Ant, B for Bear, C for Cat.
- Reward stars or stickers.
- Parent settings for voice speed and theme.
- PWA install so it can open like an app.

## 15. Copy-Paste Coding Agent Instruction

Build a simple Next.js + TypeScript + Tailwind CSS browser typing game for a child.

The app should be client-side only. No backend, database, login, analytics, or complex setup.

Core behavior:

- Listen for keyboard key presses on the main page.
- Display the actual pressed key in very large colorful text.
- Speak the pressed key aloud using the browser SpeechSynthesis API.
- Convert numbers and symbols into friendly spoken text.
- Change background/letter colors randomly on each key press.
- Show a random emoji and encouragement message.
- Keep the last 10 pressed keys visible.
- Add Mute/Unmute, Clear, and Fullscreen buttons.

Use this structure:

```
src/app/page.tsx
src/components/TypingGame.tsx
src/components/BigKeyDisplay.tsx
src/components/RecentKeys.tsx
src/components/ControlButtons.tsx
src/lib/getSpeechText.ts
src/lib/getDisplayText.ts
src/lib/random.ts
src/lib/constants.ts
```

Keep the code simple, clean, readable, and easy to extend later.